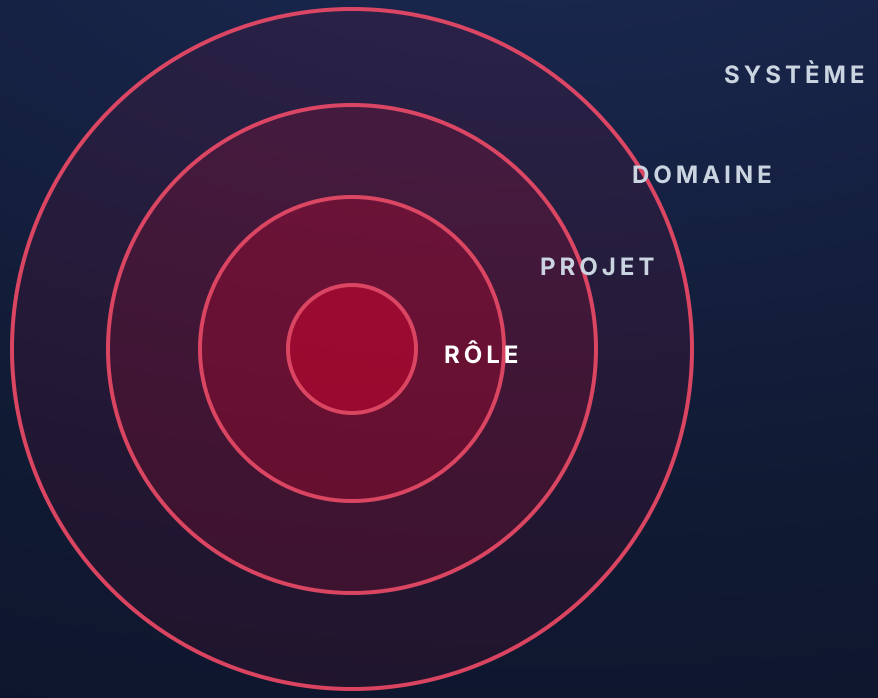
COMPRENDRE OPENSTACK · MODULE 12

Rôles et moindre privilège

Rôles standards, contrôle d'accès par périmètre (Secure RBAC) et comptes de service sur Canonical OpenStack (Sunbeam)



reader · member · manager

admin · service

Portées

Politiques

Comptes de service

Identifiants d'application

Version déployée : Canonical OpenStack 2026.1 LTS, donc OpenStack 2026.1 (les modules 1 à 5 suivent l'amont 2026.2) · documentations consultées en octobre 2026

INTRODUCTION

Objectif et déroulé

Objectif : donner à chaque personne et à chaque programme exactement les droits dont il a besoin, sur exactement le périmètre où il en a besoin, pour la durée où il en a besoin — avec les rôles standards d'OpenStack, sans réécrire les politiques.

À l'issue de ce module, vous saurez :

- Énoncer le principe de **moindre privilège** et le traduire en affectations de rôles.
- Distinguer les cinq rôles standards : `reader`, `member`, `manager`, `admin`, `service`.
- Expliquer ce que **Secure RBAC** change : portées, nouvelles règles par défaut, fin de « admin partout ».
- Lire une règle de politique et diagnostiquer un refus 403.
- Créer des **comptes de service** et des **identifiants d'application** restreints, et les faire tourner.
- Bâtir et auditer une matrice de rôles pour une organisation.

1 · Le moindre privilège
Principe, leviers, risques, histoire de Secure RBAC.

2 · Les rôles standards
Cinq rôles, implications, matrice, rôles personnalisés.

3 · Le contrôle par périmètre
Portées, personas, règles, options, diagnostic.

4 · Les comptes de service
Comptes des services, comptes d'automates, identifiants d'application, délégations.

5 · Mise en œuvre
Matrice type, pas à pas sur Sunbeam, audit, exercices.

01

Le principe de moindre privilège

Chaque droit accordé est une porte ouverte. Le moindre privilège consiste à n'ouvrir que les portes utiles — et OpenStack fournit, depuis Secure RBAC, les outils pour le faire sans bricolage.

- Le principe et ses corollaires
- Ce que coûte un privilège de trop
- Avant Secure RBAC
- Quatre leviers dans OpenStack
- La chaîne de décision
- Le calendrier de Secure RBAC

1 · LE MOINDRE PRIVILÈGE

Le principe et ses corollaires

Moindre privilège : un acteur — personne ou programme — ne détient que les droits nécessaires à sa tâche, sur le seul périmètre de cette tâche, pendant la seule durée de cette tâche.

Besoin d'en connaître

Lire est aussi un privilège. Un auditeur n'a pas besoin d'écrire ; un développeur n'a pas besoin de voir la production d'une autre équipe.

Séparation des tâches

Celui qui accorde les droits n'est pas celui qui les utilise ; celui qui déploie n'est pas celui qui valide.

Durée limitée

Un droit temporaire porte une date de fin. Un identifiant d'automate expire et se renouvelle.

Traçabilité

Un acteur, une identité : jamais de compte partagé. Sinon, impossible de savoir qui a fait quoi.

Question à poser	Mauvaise réponse	Bonne réponse
Que doit-il faire ?	**« un peu de tout, on verra »**	une liste d'opérations : lire, créer des instances, gérer des comptes…
Où ?	**« sur le cloud »**	un projet, une branche de projets, un domaine
Combien de temps ?	**« jusqu'à nouvel ordre »**	une date, ou une revue périodique qui le reconduit
Qui l'a décidé ?	**« c'était plus simple »**	un responsable nommé, une demande tracée

Quatre leviers dans OpenStack

Affectation de rôle = acteur + rôle + cible

+ durée de la preuve

① Qui ? — l'acteur

- un groupe plutôt qu'une personne
- un compte nominatif, jamais partagé
- un identifiant par automate

levier : ne donner qu'aux bons

② Quoi ? — le rôle

- reader avant member
- member avant manager
- admin : jamais par défaut

levier : le plus petit rôle

③ Où ? — la cible

- un projet plutôt qu'un domaine
- héritage seulement s'il le faut
- portée système : exploitation

levier : le plus petit périmètre

④ Combien de temps ?

- jeton : quelques heures
- identifiant d'application : daté
- affectation : revue périodique

levier : une date de fin

Le seul objet qui donne des droits est l'affectation de rôle : acteur + rôle + cible. Le moindre privilège se joue donc sur chacun de ces trois termes, plus la durée de vie de la preuve d'identité.

Ce que les leviers ne font pas : ils n'isolent pas le matériel ni le réseau. Un droit bien taillé ne remplace ni les groupes de sécurité, ni les quotas.

1 · LE MOINDRE PRIVILÈGE

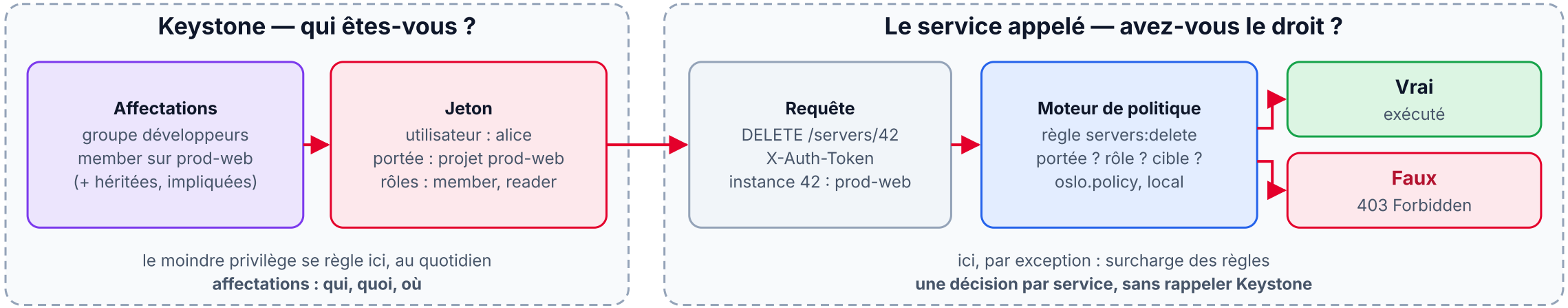
Ce que coûte un privilège de trop

Situation	Cause	Conséquence possible
Un développeur supprime une base de production	**rôle `member` sur tous les projets « pour dépanner »**	perte de données, arrêt de service
Fuite d'un fichier `openrc` d'administration	**mot de passe `admin` dans un dépôt, un poste, un ticket**	tout le cloud est à la main de l'attaquant
Le pipeline d'intégration est compromis	**il utilise le compte personnel d'un administrateur**	l'attaquant hérite de tous les droits de cette personne
Un prestataire parti garde l'accès	**compte nominatif sans date de fin, jamais revu**	accès durable hors de tout contrôle
Un gestionnaire de domaine se donne `admin`	**politique surchargée sans précaution**	élévation de privilège : sortie du domaine
Un auditeur modifie par erreur un groupe de sécurité	**`member` accordé là où `reader` suffisait**	exposition non voulue d'un service

Le coût d'un incident est proportionnel aux droits du compte compromis, pas à l'intention de son titulaire. Réduire les droits, c'est réduire le rayon d'impact.

Le moindre privilège coûte aussi : plus d'affectations, plus de demandes. Les groupes, l'héritage et l'automatisation existent pour que ce coût reste faible.

La chaîne de décision



- **Keystone décide qui vous êtes et quels rôles vous avez, sur quelle portée.** Il met tout cela dans le jeton.
- **Chaque service décide seul** si ces rôles suffisent pour l'opération : Nova applique ses règles, Neutron les siennes.
- **Conséquence :** un rôle n'a aucun pouvoir propre. `member` ne signifie quelque chose que parce que les règles des services le citent.
- **Le moindre privilège se règle à deux endroits :** les affectations, côté Keystone — le quotidien ; les règles, côté services — l'exception.

Avant Secure RBAC : « admin quelque part, admin partout »

Le modèle historique ne connaissait que deux niveaux :

Profil	Ce qu'il permettait
Propriétaire	un rôle quelconque sur le projet : lire et modifier toutes ses ressources
Administrateur	`admin` sur un projet — n'importe lequel — et les services le traitaient en administrateur du cloud entier

- **Pas de lecture seule** utilisable : tout membre d'un projet pouvait tout modifier.
- **Pas de délégation propre :** pour gérer des comptes, il fallait `admin`, donc tout.
- **Des services s'appelant entre eux avec `admin`** : un compte de service compromis valait un administrateur.

> Ce comportement a fait l'objet d'un signalement de sécurité ancien et célèbre de Keystone : le rôle `admin` n'était pas « cantonné » à son projet.

Ce que Secure RBAC apporte

Besoin	Réponse
Lecture seule	`reader`, respecté par les règles par défaut
Travailler dans un projet	`member`
Déléguer la gestion	`manager` sur un domaine ou un projet
Appels entre services	`service`, hors hiérarchie
Cibler le bon périmètre	portées : projet, domaine, système
Activer sans tout casser	deux options par service : nouvelles règles, contrôle de portée

> **Ce qui n'a pas changé :** le rôle `admin` reste global dans la plupart des services — un `admin` sur un projet administre encore le cloud. C'est le point le plus important de ce module.

1 · LE MOINDRE PRIVILÈGE

Le calendrier de Secure RBAC

Version	Étape	Effet pour l'exploitant
Rocky (2018)	**`admin`, `member`, `reader` créés par défaut dans Keystone**	les rôles existent ; les services ne les distinguent pas encore
Wallaby → Zed	**nouvelles règles par défaut dans les services, activables à la demande**	option `enforce_new_defaults` à poser soi-même
2023.1	**objectif recentré sur la portée projet ; Nova active scope et nouvelles règles par défaut**	la portée système n'ouvre plus « toutes les API »
2023.2	**rôles `manager` et `service` introduits**	délégation et appels entre services sans `admin`
2024.2	**oslo.policy 4.4 : `enforce_scope` et `enforce_new_defaults` à vrai par défaut**	nouveau comportement partout, sauf si le déploiement les remet à faux
2025.2	**persona `manager` de projet dans Nova**	opérations « gestionnaire » sur un projet, sans `admin`
2026.2	**Nova rend le contrôle de portée obligatoire**	plus de retour arrière possible dans Nova

Canonical OpenStack 2026.1 embarque des bibliothèques où les deux options valent vrai par défaut. Ce que chaque charm en fait réellement est à vérifier sur votre déploiement — voir la partie 3.

Dates reconstituées d'après l'objectif communautaire « Consistent and Secure Default RBAC » et les notes de version d'oslo.policy et de Nova ; l'objectif lui-même a été révisé plusieurs fois.

02

Les rôles standards

Cinq rôles, créés et compris par tous les services. Les utiliser tels quels est la façon la plus sûre d'appliquer le moindre privilège : leurs règles sont écrites, testées et maintenues par la communauté.

- Cinq rôles, une hiérarchie
- admin : le rôle qui déborde
- Matrice rôle × opération
- reader, member, manager
- service : le rôle des machines
- Rôles impliqués et rôles personnalisés

Cinq rôles standards

reader

Lire. Lister les instances, voir les volumes, consulter les quotas. Aucune modification.

Pour : audit, support, supervision, développeurs en production.

member

Agir sur les ressources de la cible : créer, modifier, supprimer.

Pour : les équipes qui travaillent dans un projet, les automates de déploiement.

manager

Gérer : comptes, groupes, projets et affectations dans un domaine ; certaines opérations élevées dans un projet.

Pour : responsables délégués.

admin

Tout, y compris l'infrastructure : hyperviseurs, réseaux externes, quotas, domaines.

Pour : la poignée de personnes qui exploite la plateforme.

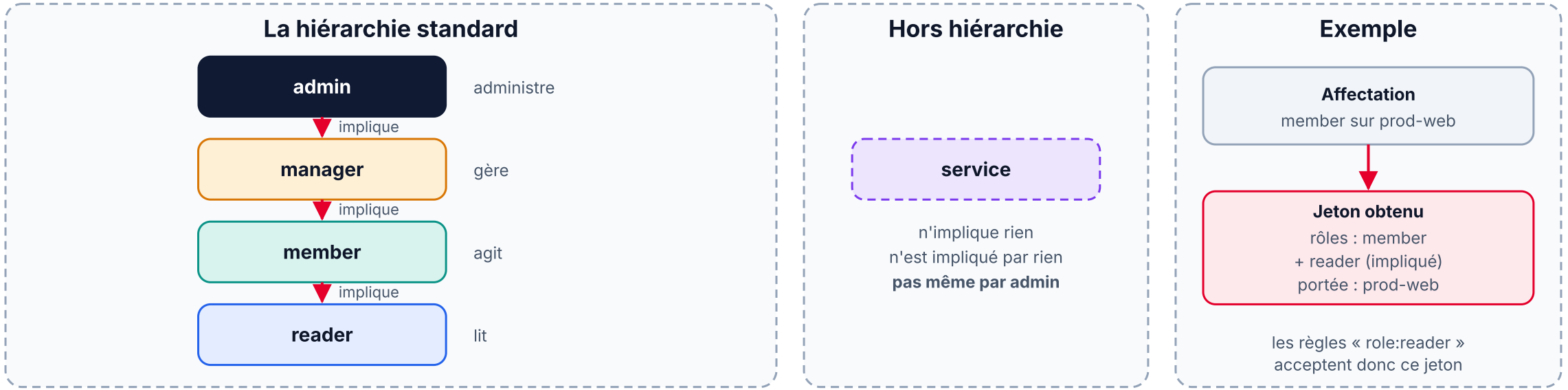
service

Appels d'un service OpenStack à un autre : notifications d'événements, opérations internes.

Pour : les comptes des services, jamais des humains.

Rôle	Créé par	Implique	Remarque
`admin`	**initialisation de Keystone**	`manager`, donc tous les autres	global : voir slide 16
`manager`	**initialisation (depuis 2023.2)**	`member`, `reader`	ne peut pas attribuer `admin`
`member`	**initialisation**	`reader`	le rôle de travail par défaut
`reader`	**initialisation**	—	base de la hiérarchie
`service`	**initialisation (depuis 2023.2) ; attribué par le charm sur Sunbeam**	— : hors hiérarchie	`admin` ne l'implique pas

La hiérarchie d'implication



- **Un rôle impliqué est ajouté au jeton automatiquement :** qui a `member` reçoit aussi `reader` dans son jeton, sans affectation supplémentaire.
- **Une seule affectation par personne et par cible suffit :** le rôle le plus haut nécessaire.
- `service` **est volontairement isolé :** un administrateur humain ne peut pas se faire passer pour un service, et un compte de service n'hérite pas de droits humains.
- **Les règles peuvent donc s'écrire simplement :** « `role:reader` » suffit pour une lecture — tous les rôles supérieurs y satisfont.

reader : voir sans toucher

Opération	Avec `reader` sur le projet
Lister, afficher instances, volumes, réseaux	oui
Consulter quotas et consommation	oui
Voir la console d'une instance	non : c'est une prise en main
Créer, arrêter, supprimer une instance	non
Modifier un groupe de sécurité	non
Télécharger une image du projet	selon le service et sa règle

Les règles de lecture vérifient aussi la cible : `role:reader and project_id:%(project_id)s` . Un `reader` sur un projet ne lit que ce projet.

Usages types

- **Audit et conformité :** `reader` hérité sur un domaine ou une branche, porté par un groupe « audit ».
- **Support de premier niveau :** voir l'état d'une ressource avant d'escalader.
- **Développeurs face à la production :** diagnostiquer sans pouvoir corriger à chaud.
- **Supervision et inventaire :** l'outil qui collecte l'état du cloud n'a besoin que de lire.

Lire n'est pas anodin : métadonnées d'instance, données utilisateur, noms et adresses révèlent l'architecture. Le besoin d'en connaître vaut aussi pour `reader` .

Dans Horizon, un `reader` voit les pages du projet ; les boutons d'action échouent avec un message d'autorisation.

member : travailler dans un projet

Opération	Avec `member` sur le projet
Créer, redimensionner, supprimer une instance	oui
Créer réseaux, routeurs, groupes de sécurité	oui
Créer et attacher des volumes	oui
Supprimer la ressource d'un collègue	oui : la ressource appartient au projet
Choisir l'hôte d'une instance	non : `admin`
Modifier les quotas du projet	non : `admin`
Ajouter un collègue au projet	non : `manager` ou `admin`
Créer un réseau partagé ou externe	non : `admin`

- **Le rôle de travail par défaut** pour les équipes et les automates de déploiement.
- **Pas de propriété individuelle :** tous les `member` d'un projet peuvent modifier toutes ses ressources. Pour séparer deux équipes, il faut deux projets, pas deux rôles.
- **Les exceptions par propriétaire** existent ponctuellement — paires de clés, identifiants d'application — mais rien de général.

```
# Prérequis : identifiants d'administrateur
source admin-openrc
openstack role add member \
  --group equipe-web --group-domain nord \
  --project rec-web --project-domain nord

# Vérifier : ce que le groupe obtient réellement
openstack role assignment list --effective \
  --group equipe-web --group-domain nord --names
```

> Le jeton d'un `member` contient `member` et `reader` : les règles de lecture lui sont acquises.

 Délégation détaillée : module 9

manager : déléguer la gestion

Sur un domaine — avec un jeton de portée domaine

Opération	Domain manager
Créer, désactiver des utilisateurs du domaine	**oui**
Gérer groupes et appartenances	**oui**
Créer, désactiver des projets du domaine	**oui**
Attribuer `reader` , `member` , `manager`	**oui, dans le domaine**
Attribuer `admin`	**non : pas d'élévation**
Modifier le domaine lui-même	**non**
Voir les autres domaines	**non**
Changer les quotas	**non**

> La liste des rôles qu'un `manager` peut attribuer est fixée par la politique de Keystone ; elle exclut `admin` et peut être étendue à un rôle maison.

Sur un projet — avec un jeton de portée projet

- **Dans Nova, depuis 2025.2 :** des opérations jusque-là réservées à `admin` deviennent accessibles au `manager` du projet — par exemple migrer à froid une instance sans choisir l'hôte.
- **Dans les autres services :** l'adoption du persona est progressive. À vérifier service par service avant de s'en servir.
- **Jamais de choix d'hôte, jamais d'infrastructure :** le `manager` reste dans son projet.

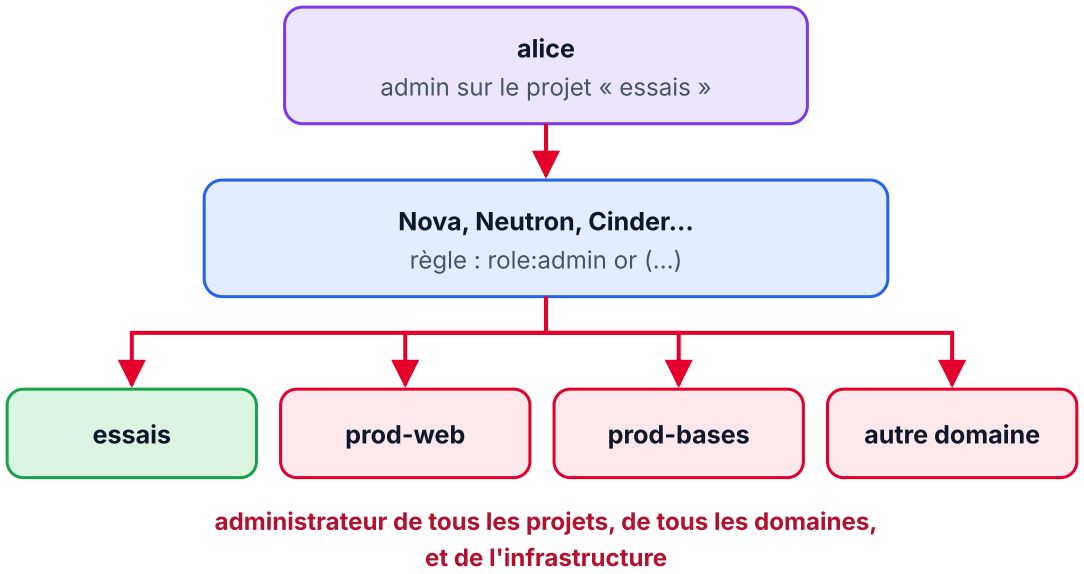
```
# Responsable du domaine « nord », délégué
openstack role add manager \
  --group gestionnaires-nord --group-domain nord \
  --domain nord

# Il travaille avec un jeton de portée domaine
export OS_DOMAIN_NAME=nord
unset OS_PROJECT_NAME OS_PROJECT_DOMAIN_NAME
openstack user create --domain nord alice
```

> Le module 9 détaille l'administration déléguée : ce que voit et ne voit pas un gestionnaire.

 Le piège admin : module 9

admin : le rôle qui déborde de sa cible



- **Dans Nova, Neutron, Cinder...** `admin` sur n'importe quel projet donne les droits d'administration sur tout le cloud. C'est le « legacy admin », conservé volontairement pour ne pas casser les déploiements existants.
- **Dans Keystone aussi :** les règles commencent par `admin_required` (`role:admin or is_admin:1`), sans condition de domaine. Un `admin` de domaine peut donc agir hors de son domaine — à tester (module 9, slide 23).
- **Conséquence pratique :** `admin` ne se délègue jamais. Pour déléguer, `manager` .

Bonne pratique	Pourquoi
Deux à cinq personnes nommées	surface minimale, imputabilité
Comptes d'administration distincts des comptes de travail	on ne lit pas ses courriels en `admin`
Affectation sur le seul projet d'administration	une seule affectation à surveiller
Revue mensuelle de la liste	retrait des départs

Règle typique de Nova :
`role:admin or (role:member and project_id:%(project_id)s)` . La partie `admin` ne vérifie pas le projet : elle vaut pour tous.

service : le rôle des machines

Certaines API ne sont faites que pour les services entre eux. Exemple : Neutron prévient Nova qu'un port est prêt ; Cinder prévient Nova qu'un volume a changé de taille.

Avant 2023.2	Avec le rôle `service`
le compte de Neutron avait `admin` pour appeler Nova	il a `service` : la règle exige ce rôle précis
un humain `admin` pouvait appeler ces API internes	un humain `admin` n'a pas `service` : refus
compromettre un compte de service = administrateur	objectif : compte de service limité aux API internes

Règle de la documentation de Keystone : `"identity:create_foo": "role:service"`, ou `"role:service or role:admin"` pendant la transition.

- **Hors hiérarchie :** ni `admin` ne l'implique, ni il n'implique `reader`.
- **Ne jamais l'attribuer à une personne** : il donne accès à des API dont l'appel à tort peut détruire des ressources.
- **Sur Sunbeam,** le charm de Keystone crée les comptes des services avec `service` **et** `admin` : le second est gardé tant que tous les services ne savent pas se contenter du premier.

```
# Qui détient le rôle service ?
openstack role assignment list --role service --names

# Attendu : uniquement des comptes du domaine
# des services, jamais une personne
```

Le jour où les comptes de service n'ont plus que `service`, un vol de leurs identifiants ne donnera plus l'administration du cloud — c'est la cible de la phase 2 de Secure RBAC.

Matrice rôle × opération

Opération (jeton de portée projet, sauf mention)	reader	member	manager	admin	service
Lister les instances du projet	oui	oui	oui	oui, tous projets	—
Créer, supprimer une instance	non	oui	oui	oui	—
Migrer à froid sans choisir l'hôte (Nova 2025.2+)	non	non	oui	oui	—
Migrer à chaud, choisir l'hôte	non	non	non	oui	—
Lister les hyperviseurs	non	non	non	oui	—
Créer un réseau, un routeur du projet	non	oui	oui	oui	—
Créer un réseau externe ou partagé	non	non	non	oui	—
Modifier les quotas	non	non	non	oui	—
Créer un utilisateur (jeton de portée domaine)	non	non	oui, son domaine	oui	—
Attribuer `admin`	non	non	non	oui	—
Événements externes de Nova (appel de Neutron)	non	non	non	selon la version	oui

Lecture : d'après les règles par défaut de Nova, Neutron et Keystone. Une surcharge de politique sur votre cloud peut changer n'importe quelle case.

Vérifier une case : lire la règle effective du service (partie 3) plutôt que de faire confiance à un tableau — celui-ci compris.

Rôles impliqués : lire et modifier la hiérarchie

```
# Prérequis : identifiants d'administrateur
source admin-openrc

# Les rôles existants
openstack role list

# Les implications : qui entraîne qui
openstack implied role list

# Ajouter une implication (exemple : rôle maison)
openstack role create facturation
openstack implied role create facturation \
  --implied-role reader

# La retirer
openstack implied role delete facturation \
  --implied-role reader
```

Résultat attendu de la deuxième commande : `admin → manager`, `manager → member`, `member → reader`. Une ligne manquante signale une installation ancienne, mise à jour sans recréer les implications.

Règle	Raison
Ne pas modifier les implications standards	**les règles des services les supposent**
Pas de cycle	**illisible, et source d'erreurs**
Pas d'implication vers `admin`	**élévation déguisée ; Keystone l'interdit par défaut (`prohibited_implied_role`)**
Pas d'implication vers `service`	**il doit rester isolé**

- **Rôle propre à un domaine :** `openstack role create --domain nord …`. Il n'apparaît jamais dans un jeton ; il ne sert qu'à impliquer des rôles globaux. Utile pour donner un nom local à une combinaison.
- **Vérifier ce que donne réellement un rôle :** `openstack role assignment list --effective --names` affiche les rôles après implication.

Rôles personnalisés : à n'utiliser qu'en dernier recours

Un rôle maison ne fait rien tant qu'aucune règle ne le cite. Le créer impose de surcharger les politiques des services concernés.

Coût	Détail
Surcharges à écrire	**une règle par opération concernée, par service**
Surcharges à maintenir	**à chaque montée de version, les règles par défaut évoluent ; les surcharges, non**
Comportement imprévu	**Horizon, les clients, les autres services ignorent le rôle**
Sur Sunbeam	**les fichiers de politique sont produits par les charms : une surcharge passe par leur configuration, pas par une édition dans le pod**

Avant de créer un rôle, essayer dans l'ordre :

#	Solution
1	un rôle standard sur une cible plus étroite — un projet plutôt qu'un domaine
2	un projet supplémentaire, pour isoler ce qui doit l'être
3	un identifiant d'application avec règles d'accès, pour un automate
4	un rôle maison qui n'implique qu'un rôle standard, pour nommer un usage
5	en dernier : un rôle maison et des surcharges de politique

Exemple légitime : un rôle `support-reseau` autorisé à modifier les groupes de sécurité de tous les projets d'un domaine, sans rien d'autre. Rien de standard ne l'offre ; la surcharge est alors justifiée — et documentée.

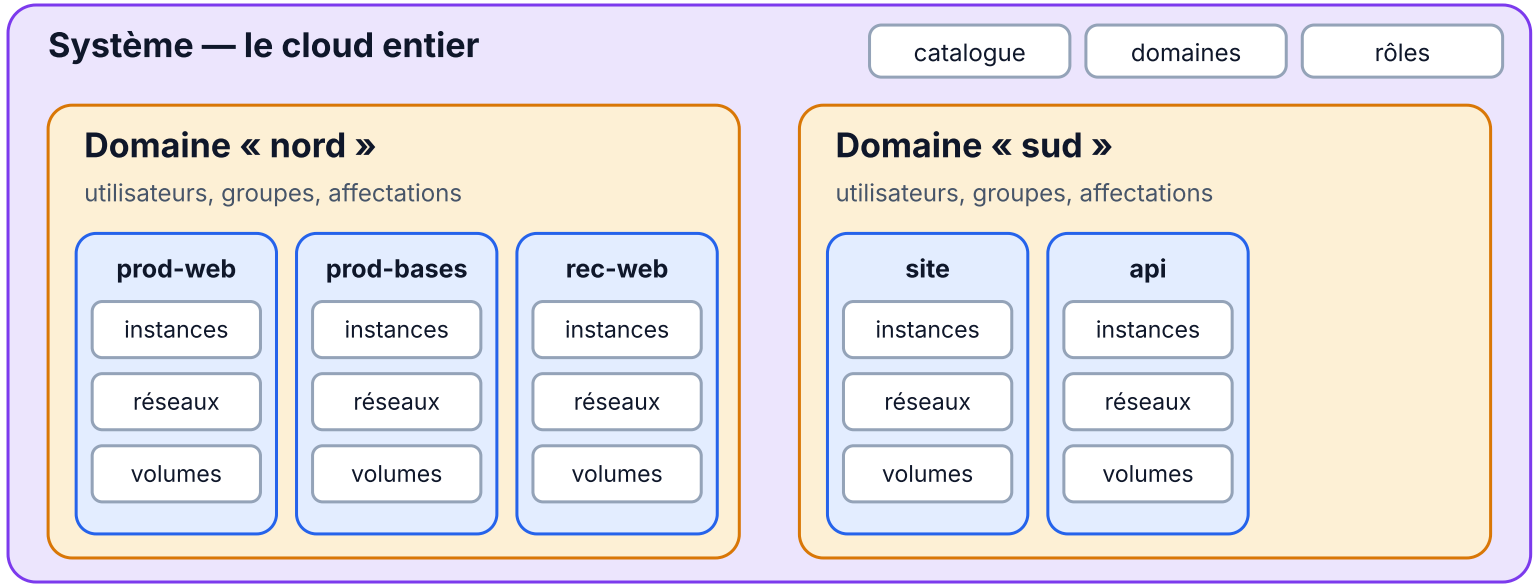
03

Le contrôle d'accès par périmètre — Secure RBAC

Un rôle ne vaut que sur une cible. Secure RBAC rend cette cible explicite : chaque jeton a une portée, chaque règle dit quelles portées elle accepte et vérifie que la ressource visée en fait partie.

- Trois portées
- Anatomie d'une règle
- Système, domaine, projet
- Lire et surcharger les règles
- L'état de votre cloud Sunbeam
- Personas : rôle × portée
- Les deux options d'activation
- Choisir la portée de son jeton
- Diagnostiquer un refus

Trois portées



Jeton et ce qu'il ouvre

jeton système
`OS_SYSTEM_SCOPE=all`
identité de tout le cloud

jeton de domaine
`OS_DOMAIN_NAME=nord`
identité du domaine nord

jeton de projet
`OS_PROJECT_NAME=prod-web`
ressources de prod-web

Système
Ce qui n'appartient à personne : catalogue, domaines, rôles, et dans certains services l'infrastructure. Réservée à l'exploitation.

Domaine
L'identité d'une organisation : ses utilisateurs, groupes, projets, affectations. Portée de la délégation.

Projet
Les ressources : instances, réseaux, volumes, images. La portée de presque tout le travail quotidien.

Personas : un rôle sur une portée

	Portée système	Portée domaine	Portée projet
`admin`	**administre Keystone : catalogue, domaines, fédération**	administre l'identité — et déborde du domaine (`admin_required`)	« legacy admin » : administre tous les services, tous projets
`manager`	—	gère utilisateurs, groupes, projets, affectations du domaine	opérations élevées dans le projet, là où le service le prévoit
`member`	**lit l'identité de tout le cloud**	lit le domaine	crée et gère les ressources du projet
`reader`	**lit l'identité de tout le cloud — audit**	lit le domaine	lit les ressources du projet

- **Un persona = un couple.** « Lecteur système » et « lecteur de projet » sont deux profils différents, obtenus par deux affectations différentes.
- **Les cases de la colonne système** concernent Keystone. Les services de ressources — Nova en tête — n'acceptent plus de jeton système : leurs règles sont de portée projet.
- **Les droits ne se transportent pas d'une portée à l'autre :** un `reader` système ne lit pas les instances ; il lui faut un jeton de projet.
- **Exception :** `admin`, global (slide 16).

Anatomie d'une règle

```
# Nova — règles par défaut (extraits)
"project_reader":  "role:reader and project_id:%(project_id)s"
"project_member":  "role:member and project_id:%(project_id)s"
"project_manager_api":
                   "role:manager and project_id:%(project_id)s"
"admin_api":       "role:admin"

"os_compute_api:servers:show":
  "role:admin or (role:reader and project_id:%(project_id)s)"
"os_compute_api:servers:create":
  "role:admin or (role:member and project_id:%(project_id)s)"
"os_compute_api:os-hypervisors:list":
  "role:admin"

# Keystone — lire les utilisateurs
"identity:list_users":
  "(rule:admin_required)
   or (role:reader and system_scope:all)
   or (role:reader and domain_id:%(target.domain_id)s)"
```

Élément	Ce qu'il vérifie
`role:member`	le jeton contient ce rôle, directement ou par implication
`project_id:%(project_id)s`	la ressource visée appartient au projet du jeton
`domain_id:%(target.domain_id)s`	la cible est dans le domaine du jeton
`system_scope:all`	le jeton est de portée système
`rule:…`	renvoie à une autre règle nommée
Portées acceptées	attribut de chaque règle dans le code : projet, domaine, système ; non surchargeable par fichier dans Nova

Trois contrôles successifs : la portée du jeton est-elle acceptée par cette opération ? le rôle est-il présent ? la cible appartient-elle à la portée ? Il faut les trois.

`role:admin` **seul, sans condition de projet :** c'est là que se loge le « legacy admin ».

Les deux options d'activation

`[oslo_policy] enforce_new_defaults`

Vrai : seules les nouvelles règles par défaut — avec `reader`, `member`, `manager` — s'appliquent. Faux : les anciennes règles restent acceptées en alternative, et tout rôle sur un projet suffit souvent à y écrire.

`[oslo_policy] enforce_scope`

Vrai : un jeton dont la portée ne figure pas parmi celles de la règle est refusé (403). Faux : la portée n'est que signalée dans les journaux.

Combinaison	Comportement	Quand
faux · faux	**comportement historique : `reader` peut écrire, jeton système accepté partout**	clouds anciens, à faire évoluer
vrai · faux	**rôles respectés, portées seulement journalisées**	étape de migration : observer les refus de portée à venir
faux · vrai	**combinaison incohérente ; à éviter**	—
vrai · vrai	**Secure RBAC complet**	cible ; valeur par défaut depuis oslo.policy 4.4 (2024.2)

Une option par service : chaque service lit son propre fichier de configuration. Un cloud peut être en Secure RBAC complet dans Nova et en mode historique dans un autre service.

Nova recommande d'activer les deux ensemble, et rend le contrôle de portée obligatoire en 2026.2 : l'option n'y aura plus d'effet.

La portée système : ce qu'il en reste

L'idée de départ : un jeton système pour tout ce qui touche à l'infrastructure, dans tous les services. **Le résultat** : l'objectif communautaire a été recentré sur la portée projet en 2023.

Service	Jeton de portée système
Keystone	**utilisé : catalogue, domaines, rôles, fédération, lecture de toute l'identité**
Ironic	**conserve sa propre mise en œuvre des portées**
Nova	**refusé : toutes les règles sont de portée projet ; un jeton système reçoit un 403**
Autres services	**en général portée projet ; à vérifier service par service**

> L'administration des ressources se fait donc avec `admin` sur un projet — le projet d'administration — et non avec un jeton système.

```
# Donner la lecture de toute l'identité à l'audit
openstack role add reader \
  --group audit --group-domain nord --system all

# Obtenir un jeton système
export OS_SYSTEM_SCOPE=all
unset OS_PROJECT_NAME OS_PROJECT_DOMAIN_NAME OS_DOMAIN_NAME
openstack user list          # accepté
openstack server list        # 403 dans Nova
```

- **Affectation système :** visible avec `openstack role assignment list --system all --names`.
- **À limiter** aux équipes plateforme et d'audit : le lecteur système voit tous les comptes de tous les domaines.
- **Ne pas confondre** avec le projet d'administration : ce sont deux cibles distinctes, deux affectations distinctes.

La portée domaine : la délégation

- **Un jeton de portée domaine** ne donne accès à aucune ressource : pas d'instance, pas de réseau. Il sert à gérer l'identité du domaine.
- **Les règles de Keystone** comparent le domaine du jeton à celui de la cible : `domain_id:%(target.domain_id)s`. Un gestionnaire du domaine « nord » reçoit 403 sur un utilisateur de « sud ».
- **Pour agir dans les projets du domaine,** le gestionnaire a besoin en plus d'un rôle sur ces projets — direct, ou hérité depuis le domaine — et d'un jeton de portée projet.

Persona	Usage
`manager` **sur le domaine**	**responsable délégué de l'organisation**
`reader` **sur le domaine**	**audit des comptes et affectations du domaine**
`admin` **sur le domaine**	**à éviter : ses jetons de projet seraient ceux d'un administrateur du cloud**

```
# Jeton de portée domaine
export OS_DOMAIN_NAME=nord
unset OS_PROJECT_NAME OS_PROJECT_DOMAIN_NAME OS_SYSTEM_SCOPE
openstack token issue

# Ce qu'un gestionnaire du domaine peut faire
openstack user create --domain nord \
  --password-prompt bob
openstack project create --domain nord essais
openstack role add member --user bob \
  --user-domain nord --project essais --project-domain nord

# Ce qui lui est refusé
openstack role add admin --user bob \
  --user-domain nord --project essais --project-domain nord
# 403 : admin n'est pas dans sa liste
openstack user list --domain sud      # 403
```

> « Rôle hérité sur le domaine » et « rôle sur le domaine » sont deux affectations différentes : la première descend dans les projets, la seconde donne la portée domaine.

3 · SECURE RBAC

Choisir la portée de son jeton

Portée	Variables d'environnement	Équivalent clouds.yaml
Projet	`OS_PROJECT_NAME`, `OS_PROJECT_DOMAIN_NAME`	`project_name`, `project_domain_name`
Domaine	`OS_DOMAIN_NAME`	`domain_name`
Système	`OS_SYSTEM_SCOPE=all`	`system_scope: all`

- **Une seule portée par jeton :** définir deux familles de variables à la fois donne un comportement imprévisible. Toujours `unset` les autres.
- **Un fichier par rôle de travail :** `projet-web.rc`, `domaine-nord.rc`, `systeme.rc` — ou une entrée par portée dans `clouds.yaml`.
- **Le jeton dit sa portée :** `openstack token issue -f yaml` affiche `project_id`, `domain_id` ou `system`.

```
# ~/.config/openstack/clouds.yaml — trois entrées,
# un même compte, trois portées
clouds:
  nord-web:
    auth:
      auth_url: https://keystone.exemple.lan/v3
      username: alice
      user_domain_name: nord
      project_name: prod-web
      project_domain_name: nord
  nord-domaine:
    auth:
      auth_url: https://keystone.exemple.lan/v3
      username: alice
      user_domain_name: nord
      domain_name: nord
  plateforme-systeme:
    auth:
      auth_url: https://keystone.exemple.lan/v3
      username: alice-adm
      user_domain_name: admin_domain  # cf. admin-openrc
      system_scope: all

# Usage
openstack --os-cloud nord-web server list
```

Lire les règles effectives

```
# Outils fournis par oslo.policy, présents là où
# le service est installé

# Règles par défaut d'un service, commentées
oslopolicy-sample-generator --namespace nova

# Règles effectives : défauts + surcharges du fichier
oslopolicy-policy-generator --namespace nova

# Surcharges devenues inutiles (identiques aux défauts)
oslopolicy-list-redundant --namespace nova

# Tester une règle contre un jeton enregistré en JSON
oslopolicy-checker --policy policy.yaml \
  --access jeton.json --rule os_compute_api:servers:create

# Sur Sunbeam : dans le conteneur du service (noms à vérifier)
sudo k8s kubectl -n openstack exec keystone-0 -c keystone -- \
  oslopolicy-policy-generator --namespace keystone
```

Où lire	Ce qu'on y trouve
Documentation de chaque service, page « policy »	**chaque règle, sa valeur par défaut, ses portées**
Générateur d'exemple	**la même chose, pour la version installée**
Générateur de règles effectives	**ce qui s'applique réellement sur ce cloud**
Journaux du service	**les avertissements de règles dépréciées ou de portée**

- **Ne jamais conclure d'après un tableau de formation :** une règle se vérifie dans la version installée.
- **Nombre de règles :** plusieurs centaines par service ; on cherche par nom d'opération — `grep servers:create`.

> Sur Sunbeam, le noyau Kubernetes est piloté par `sudo k8s kubectl`, et chaque service tourne dans le modèle Juju `openstack`.

Surcharger une règle

```
# policy.yaml de Neutron — seules les règles
# modifiées y figurent ; le reste garde ses défauts

# Rôle maison : le support réseau gère les règles
# des groupes de sécurité de tous les projets
"create_security_group_rule":
  "rule:admin_only or role:support-reseau
   or (role:member and project_id:%(project_id)s)"
"delete_security_group_rule":
  "rule:admin_only or role:support-reseau
   or (role:member and project_id:%(project_id)s)"
```

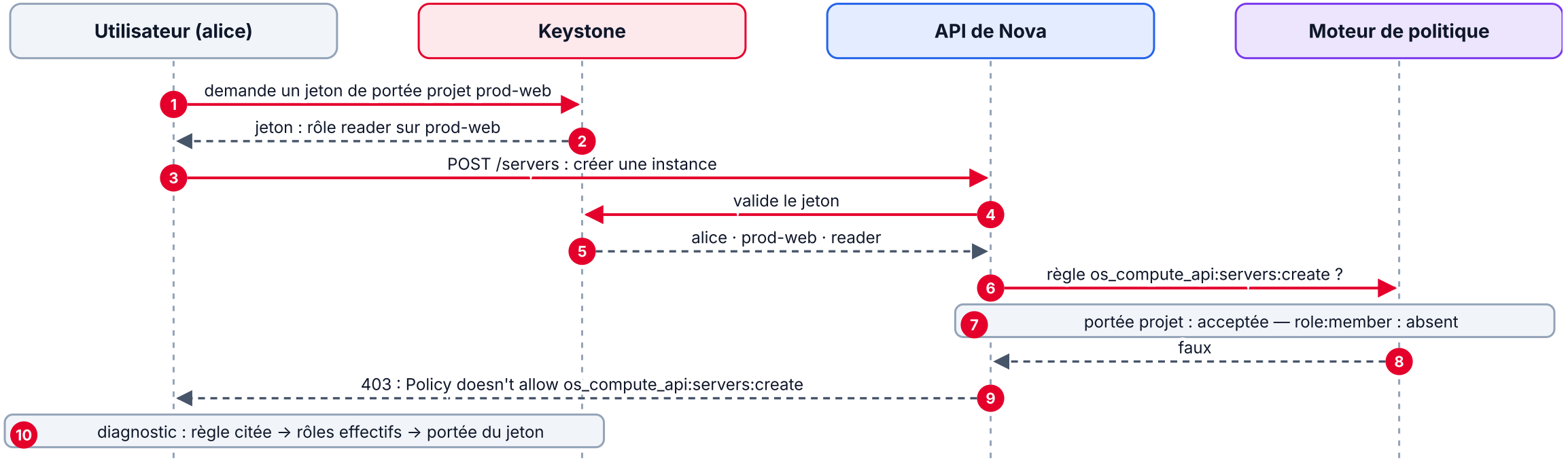
Étape	Commande ou action
1 · Partir du défaut	copier la règle du générateur d'exemple
2 · Ajouter, pas remplacer	une alternative `or`, le reste intact
3 · Tester	`oslopolicy-checker`, puis un compte réel
4 · Documenter	pourquoi, pour qui, jusqu'à quand

- **Une surcharge figée ne suit plus les évolutions :** à chaque version, comparer avec les nouveaux défauts — `oslopolicy-list-redundant` aide à retirer ce qui ne sert plus.
- **Risque d'élévation :** une alternative mal écrite — `role:support-reseau` sans condition — ouvre l'opération partout. C'est voulu ici ; ce doit l'être toujours.
- **Format :** YAML. Les fichiers JSON sont dépréciés ; `oslopolicy-convert-json-to-yaml` convertit.

Sur Sunbeam : les fichiers de configuration des services sont écrits par les charms. Une modification à la main dans un pod est perdue au prochain redémarrage ou à la prochaine mise à jour. Vérifier ce que le charm propose — `juju config -m openstack neutron` — avant toute surcharge ; à défaut, s'en passer.

Dans la très grande majorité des cas, on n'a pas besoin de surcharge : un rôle standard sur la bonne cible suffit.

Diagnostiquer un refus 403



- **Message type de Nova :** « Policy doesn't allow os_compute_api:servers:create to be performed ». Le nom de la règle est dans le message : c'est le point de départ.
- **Rôles effectifs :** `openstack role assignment list --effective --user alice --user-domain nord --names`.
- **Portée du jeton :** `openstack token issue -f yaml` ; un jeton de domaine ou système sur Nova est refusé.
- **Affectation récente :** un nouveau rôle n'apparaît que dans un nouveau jeton. Le client en redemande un à chaque commande ; Horizon, à la reconnexion.

Secure RBAC sur votre cloud Sunbeam : vérifier

```
# Prérequis : accès au nœud de contrôle Sunbeam

# 1 · Les options effectives dans chaque service
for s in keystone nova neutron cinder glance; do
  echo "== $s"
  sudo k8s kubectl -n openstack exec ${s}-0 -c ${s}-api -- \
    sh -c 'grep -rA3 "^\[oslo_policy\]" /etc/'"$s"'/' 2>/dev/null
done
# (noms de conteneurs à adapter : keystone-0 -c keystone…)

# 2 · Les rôles standards et leurs implications
source admin-openrc
openstack role list
openstack implied role list

# 3 · Essai réel : un reader ne doit pas pouvoir écrire
openstack --os-cloud lecteur-web server create \
  --flavor m1.small --image ubuntu essai
# attendu : 403 Policy doesn't allow …
```

Constat	Interprétation
Aucune section `[oslo_policy]`	valeurs par défaut de la bibliothèque : vrai · vrai en 2026.1
`enforce_new_defaults = False`	le charm a gardé l'ancien comportement : `reader` n'est pas fiable
`manager` ou `service` absents	installation ancienne : rôles à créer
Le `reader` a pu créer l'instance	nouvelles règles non appliquées dans Nova

Point à vérifier avant tout cours ou toute procédure : ce module décrit le comportement des règles par défaut. Ce que les charms de Canonical OpenStack 2026.1 posent réellement dans chaque service n'a pas été contrôlé ici ; le test n° 3 tranche en une minute.

L'essai réel est la seule preuve qui vaille : il vérifie ensemble configuration, règles et affectations.

04

Les comptes de service

Les comptes les plus puissants d'un cloud ne sont souvent pas ceux des personnes : ce sont ceux des services et des automates. Ils ne dorment jamais, ne partent jamais en congé, et leurs secrets traînent dans des fichiers.

- Deux familles de comptes non humains
- Les jetons de service
- Identifiants d'application
- Rotation et révocation
- Les comptes des services sur Sunbeam
- Comptes d'automates : quatre options
- Règles d'accès
- Délégations (trusts)

4 · COMPTES DE SERVICE

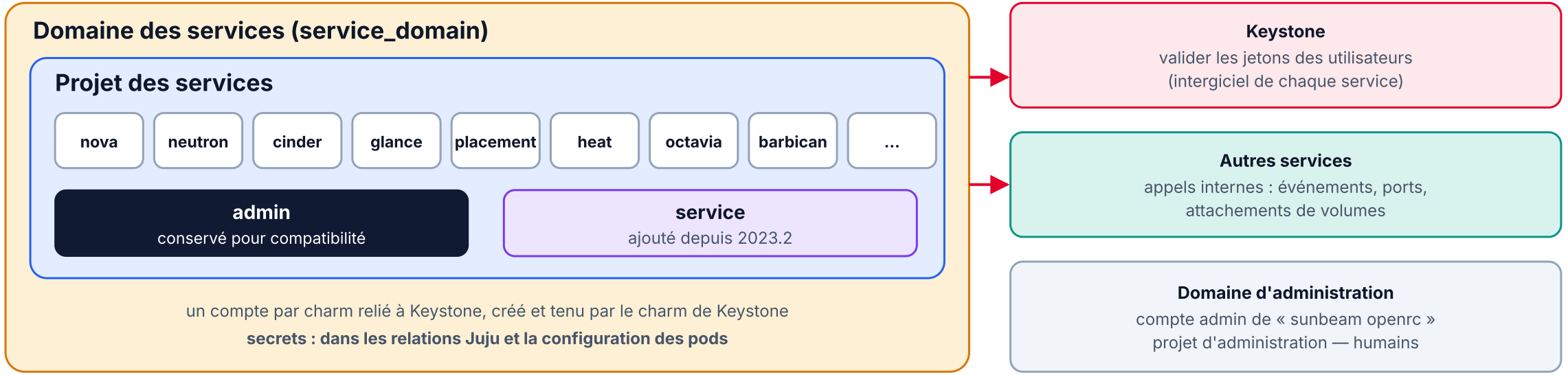
Deux familles de comptes non humains

	Comptes des services OpenStack	Comptes d'automates
Exemples	`nova`, `neutron`, `cinder`, `glance`, `placement`…	pipeline de déploiement, Terraform, sauvegarde, supervision, inventaire
Créés par	**le déploiement ; sur Sunbeam, les charms**	l'équipe plateforme, ou le gestionnaire du domaine
Servent à	**valider les jetons, appeler les autres services**	créer et gérer des ressources dans un ou plusieurs projets
Rôles	`service` **— et encore** `admin` **sur Sunbeam**	`member` ou `reader`, sur le projet strictement nécessaire
Secret	**généré et rangé par le charm ; jamais manipulé à la main**	identifiant d'application, rangé dans un coffre
Qui y touche	**personne (slide suivante)**	le propriétaire de l'automate, qui en répond

Erreur la plus fréquente : réutiliser un compte de service OpenStack — ou le compte `admin` — pour un automate « parce qu'il marche ». C'est donner l'administration du cloud à un pipeline.

Règle simple : un automate = un compte (ou un identifiant d'application) dédié, avec le plus petit rôle, sur le plus petit périmètre, avec une date d'expiration.

Les comptes des services sur Sunbeam



- **Chaque charm qui se relie à Keystone** reçoit un compte dans le domaine des services, avec les rôles `admin` et `service` sur le projet des services.
- `admin` **est conservé** tant que tous les services ne savent pas se contenter de `service` : c'est un choix de compatibilité du charm.
- **Conséquence :** les secrets de ces comptes valent un accès administrateur. Ils vivent dans les relations Juju et les fichiers de configuration des pods : protéger l'accès au nœud de contrôle et au modèle Juju.
- **Ne rien y modifier à la main** — ni mot de passe, ni rôle : le charm les recrée ou se désynchronise.

4 · COMPTES DE SERVICE Vu au module 8

Les jetons de service

Problème : Nova démarre une opération longue pour Alice — une migration, un attachement de volume — puis appelle Cinder avec le jeton d'Alice. Si ce jeton expire en route, l'opération échoue à mi-parcours.

Solution : Nova joint son propre jeton, en second en-tête.

En-tête	Contenu
`X-Auth-Token`	**le jeton de l'utilisateur : ses droits s'appliquent**
`X-Service-Token`	**le jeton du service : prouve que l'appel vient de Nova**

> Le service appelé accepte alors un jeton utilisateur expiré depuis peu, parce qu'un service de confiance l'accompagne. Les droits restent ceux de l'utilisateur : le service ne lui prête pas les siens.

```
# Côté service appelant (ex. nova.conf)
[service_user]
send_service_user_token = true
auth_type = password
username = nova
user_domain_name = service_domain
project_name = services
project_domain_name = service_domain

# Côté service appelé (keystonemiddleware)
[keystone_authtoken]
service_token_roles = service
service_token_roles_required = true
```

- `service_token_roles` : le jeton de service doit porter ce rôle — c'est l'un des usages du rôle `service`.
- **Sur Sunbeam,** ces sections sont écrites par les charms.

> Valeurs d'exemple ; les noms de domaine et de projet des services sont à lire dans la configuration réelle de votre cloud.

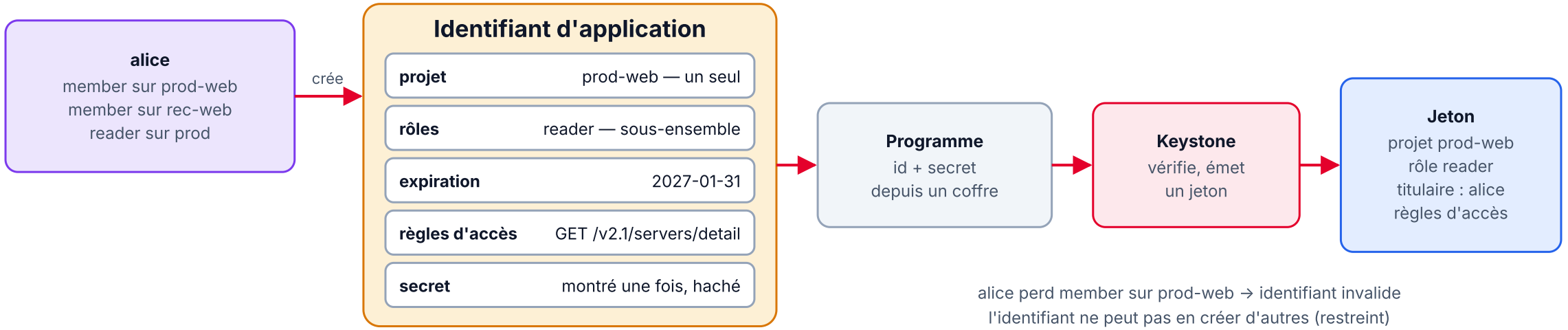
Comptes d'automates : quatre options

Critère	Compte personnel	Utilisateur technique dédié	Identifiant d'application	Délégation (trust)
Secret	**mot de passe d'une personne**	mot de passe propre	secret propre, haché par Keystone, montré une fois	celui du service délégataire
Portée	**tous les projets de la personne**	ses affectations	**un seul projet**	un projet, rôles choisis
Rôles	**tous ceux de la personne**	ceux qu'on lui donne	**sous-ensemble de ceux du créateur**	**sous-ensemble de ceux du délégant**
Restriction par API	**non**	**non**	**oui : règles d'accès**	**non**
Expiration	**celle du mot de passe**	celle du mot de passe	**date d'expiration**	**date d'expiration**
Départ du créateur	**le pipeline s'arrête**	sans effet	invalide : à faire tourner avant	invalide
Imputabilité	**confondue avec la personne**	**compte nommé**	rattaché au créateur, nommé	délégant et délégataire
Usage recommandé	**jamais**	automate de plateforme sans titulaire humain	**cas général des automates**	services qui agissent plus tard pour un utilisateur : Heat, Magnum

Compte personnel dans un pipeline : l'automate a tous les droits de la personne, sur tous ses projets, et s'arrête le jour de son départ. C'est l'anti-modèle.

Utilisateur technique ou identifiant d'application ? L'identifiant d'application est plus étroit ; l'utilisateur technique survit au départ de quiconque. On combine souvent : un utilisateur technique, qui crée ses identifiants d'application.

Identifiants d'application : principe



- **Créés par l'utilisateur lui-même,** sans intervention de l'administration : la délégation est en libre-service.
- **Le secret n'est montré qu'une fois :** Keystone le hache avant de le stocker. Perdu, il ne se récupère pas — on en crée un autre.
- **Ne peuvent jamais dépasser leur créateur :** on ne donne pas un rôle qu'on n'a pas. Retirer un rôle au créateur invalide ses identifiants sur ce projet.
- **Restreints par défaut :** ils ne peuvent pas créer ni supprimer d'autres identifiants d'application — sauf option `--unrestricted`, à éviter.

Créer un identifiant d'application restreint

```
# Prérequis : être connecté avec un jeton de portée projet,
# sur le projet visé, avec le compte qui en sera titulaire
openstack --os-cloud ci-web token issue -c project_id

# Identifiant d'un pipeline de déploiement : member,
# sur prod-web seulement, valable jusqu'à fin janvier
openstack --os-cloud ci-web application credential create \
  deploy-web-2026q4 \
  --role member \
  --expiration 2027-01-31T00:00:00 \
  --description "Pipeline GitLab web — titulaire : equipe-web"

# Identifiant d'un outil d'inventaire : lecture seule
openstack --os-cloud ci-web application credential create \
  inventaire-web --role reader \
  --expiration 2027-03-31T00:00:00

# Lister, afficher, supprimer
openstack application credential list
openstack application credential show deploy-web-2026q4
openstack application credential delete inventaire-web
```

Option	Effet
`--role`	sous-ensemble des rôles du créateur ; par défaut, tous
`--expiration`	date de fin ; sans elle, aucune expiration
`--access-rules`	liste blanche d'appels d'API autorisés
`--secret`	fournir le secret au lieu de le laisser générer — à éviter
`--unrestricted`	autorise à créer d'autres identifiants — à éviter

La réponse contient `id` et `secret` : les copier immédiatement dans le coffre de secrets du pipeline, jamais dans un dépôt, un ticket ou une messagerie.

Nom porteur de sens : usage + périmètre + période. À la revue, on sait ce qu'il fait et quand il doit tourner.

Règles d'accès : restreindre aux appels utiles

```
# Une sauvegarde qui ne fait que lister les volumes
# et créer des instantanés
openstack application credential create sauvegarde-web \
  --role member --expiration 2027-01-31T00:00:00 \
  --access-rules '[
    {"service": "volumev3", "method": "GET",
     "path": "/v3/*/volumes/detail"},
    {"service": "volumev3", "method": "POST",
     "path": "/v3/*/snapshots"},
    {"service": "volumev3", "method": "GET",
     "path": "/v3/*/snapshots/**"}
  ]'

# Règles déjà connues, réutilisables par identifiant
openstack access rule list
```

Chemins : `*` remplace un segment, `**` un nombre quelconque de segments. Le chemin part de la racine de l'adresse du service, version comprise.

Usage	Règle(s) — service, méthode, chemin
Créer des instances	`compute POST /v2.1/servers`
Lister les instances	`compute GET /v2.1/servers/detail`
Téléverser des images	`image POST /v2/images`, `image PUT /v2/images/*/file`
Lire les ports	`network GET /v2.0/ports`

- **Le rôle reste nécessaire :** la règle d'accès filtre en plus. Il faut que la politique du service **et** la règle d'accès disent oui.
- **Le service est désigné par son type** dans le catalogue — `openstack catalog list` le donne.
- **Mettre au point par essais :** lancer l'automate, relever les 401 et les chemins refusés, compléter. Les clients appellent parfois des API inattendues — découverte de versions, extensions.

Les services doivent déclarer leur type à keystonemiddleware pour que les règles s'appliquent ; à confirmer sur Sunbeam par un essai avant de s'y fier.

Utiliser un identifiant d'application

```
# clouds.yaml du pipeline (secret injecté au lancement)
clouds:
  deploy-web:
    auth_type: v3applicationcredential
    auth:
      auth_url: https://keystone.exemple.lan/v3
      application_credential_id: 6f1c…
      application_credential_secret: ${SECRET}
    region_name: RegionOne
    interface: public

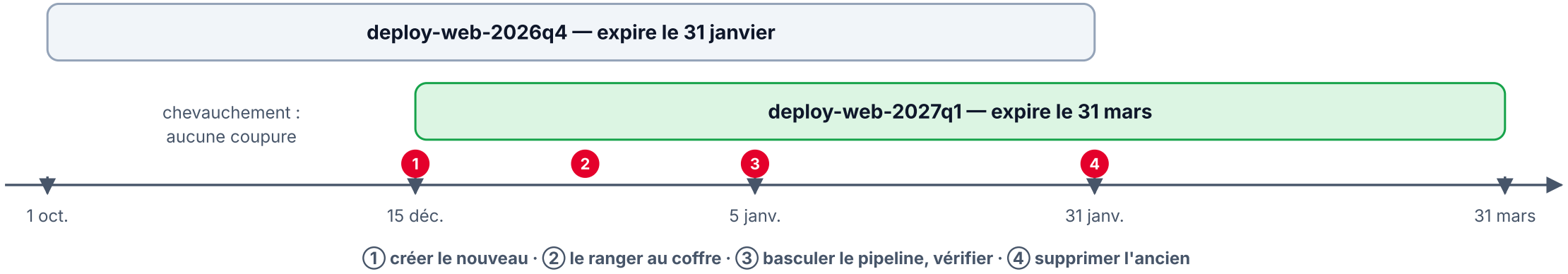
# Ou par variables d'environnement
export OS_AUTH_TYPE=v3applicationcredential
export OS_AUTH_URL=https://keystone.exemple.lan/v3
export OS_APPLICATION_CREDENTIAL_ID=6f1c…
export OS_APPLICATION_CREDENTIAL_SECRET="$SECRET"
openstack server list
```

```
# Terraform — fournisseur OpenStack
provider "openstack" {
  auth_url                      = var.auth_url
  application_credential_id     = var.app_cred_id
  application_credential_secret = var.app_cred_secret
}
```

- **Ni nom d'utilisateur, ni projet, ni domaine :** l'identifiant porte déjà le projet et les rôles. Ajouter `OS_PROJECT_NAME` provoque une erreur.
- **Le secret vient d'un coffre** — variables protégées et masquées de GitLab, coffre de secrets — au moment de l'exécution.
- **Le jeton obtenu** est un jeton de portée projet ordinaire : les règles des services s'appliquent comme pour son créateur, limitées aux rôles choisis.

> Vérifier ce qu'obtient l'automate : `openstack --os-cloud deploy-web token issue` puis un appel attendu et un appel qui doit échouer.

Rotation et révocation



- **Rotation sans coupure :** le nouvel identifiant existe avant que l'ancien disparaisse ; deux noms différents.
- **Fréquence :** à chaque trimestre, ou plus court pour la production.

- **Révocation d'urgence :** `application credential delete` — effet immédiat pour les nouveaux jetons ; les jetons déjà émis vivent jusqu'à leur expiration.
- **Pour tout couper :** retirer le rôle du titulaire sur le projet, ce qui invalide ses identifiants.

- **Départ du titulaire :** recréer les identifiants sous un autre titulaire **avant** de désactiver le compte.
- **Fuite suspectée :** révoquer d'abord, enquêter ensuite.

Délégations (trusts)

Une délégation permet à un utilisateur — le délégant — d'autoriser un autre utilisateur — le délégataire — à agir en son nom, sur un projet, avec certains de ses rôles, pour une durée donnée.

Qui s'en sert	Pour quoi
Heat	**agir plus tard au nom de l'utilisateur : mise à l'échelle automatique, mise à jour d'une pile**
Magnum	**permettre au cluster de gérer ses ressources dans le projet**
Automates internes	**rarement : on préfère les identifiants d'application**

> **Différence clé :** un identifiant d'application est un secret pour un programme ; une délégation est un lien entre deux comptes existants, que le délégataire utilise avec ses propres secrets.

```
# Alice délègue member sur prod-web au compte
# technique « exploitation-bot », pour trois mois
openstack trust create \
  --project prod-web --project-domain nord \
  --role member \
  --expiration 2027-01-13T00:00:00 \
  --trustor-domain nord --trustee-domain nord \
  alice exploitation-bot

openstack trust list
openstack trust show <id>
openstack trust delete <id>
```

- `--impersonate` : le jeton obtenu porte l'identité du délégant. Plus transparent pour les services, moins lisible dans les journaux — à éviter sauf besoin.
- **Délégations créées par les services** — Heat, Magnum — : les lister à la revue ; un délégant parti les rend inopérantes.

Comptes d'automates : règles de conduite

Règle	Mise en œuvre
Un usage, un identifiant	**déploiement, sauvegarde, inventaire : trois identifiants distincts**
Un environnement, un identifiant	**jamais le même secret en production et hors production**
Le plus petit rôle	**`reader` pour lire ; `member` seulement pour créer**
Le plus petit périmètre	**un projet ; plusieurs projets = plusieurs identifiants**
Une date de fin	**toujours `--expiration`**
Un titulaire nommé	**description : équipe, contact, ticket**
Un coffre	**secret injecté à l'exécution, masqué dans les journaux**

Interdits

- Le fichier `admin-openrc` de Sunbeam dans un pipeline, un dépôt, un poste de développeur.
- Les comptes des services OpenStack réutilisés pour autre chose.
- Un compte d'automate avec `admin` — sauf l'automate de la plateforme elle-même, isolé et surveillé.
- Un secret partagé entre plusieurs équipes.
- Des identifiants sans expiration, jamais revus.

Cas de l'automate de plateforme (Terraform qui crée domaines, projets, quotas) : il a besoin d'`admin`. On le limite autrement : exécution depuis un poste ou un exécuteur dédié, secret à part, journalisation, revue des changements avant application.

> Le meilleur secret est celui qui n'existe pas : lorsque l'outil le permet, préférer une authentification fédérée de courte durée à un secret statique.

05

Mise en œuvre sur Sunbeam

Des rôles standards, sur des cibles étroites, portés par des groupes ; des automates à identifiants dédiés et expirants ; une revue régulière. Le tout se construit avec les identifiants d'administration fournis par Sunbeam, puis se délègue.

- Une matrice de rôles type
- Pas à pas : automates et vérification
- Erreurs fréquentes
- Synthèse
- Pas à pas : groupes et affectations
- Auditer les droits
- Exercices

Une matrice de rôles type

Population	Rôle	Cible	Hérité	Commentaire
Équipe plateforme (2 à 5 personnes)	`admin`	projet d'administration	non	comptes d'administration distincts des comptes de travail
Équipe plateforme — lecture	`reader`	système	—	voir toute l'identité sans passer par `admin`
Gestionnaires du domaine nord	`manager`	domaine nord	non	comptes, groupes, projets, affectations
Exploitation de la production	`member`	projet `prod` (nœud)	oui	tous les projets de production, présents et futurs
Équipe web (`equipe-web`)	`member`	projets de recette et de développement	non	affectation directe, projet par projet
Développeurs — production	`reader`	projet `prod` (nœud)	oui	diagnostiquer sans corriger
Audit	`reader`	domaine nord	oui	et `reader` sur le domaine, pour lire les comptes
Pipeline de déploiement web	`member`	projet `prod-web`	non	identifiant d'application, expiration trimestrielle
Supervision	`reader`	chaque projet surveillé	non	identifiant d'application par projet
Services OpenStack	`admin` + `service`	projet des services	non	géré par les charms : ne pas toucher

Aucun rôle donné à une personne : toutes les affectations humaines passent par des groupes. Ajouter ou retirer quelqu'un = modifier une appartenance.

Cette matrice suppose l'arbre de projets « par environnement » du module 10 : nœuds `prod` et `horsprod`, feuilles par application.

Pas à pas (1) : groupes et affectations

```
# Étape 0 — prérequis : cloud Sunbeam déployé, client
# openstack installé, domaine nord et projets créés
sunbeam openrc > admin-openrc
source admin-openrc
openstack role list      # reader member manager admin service

# Étape 1 — les groupes
for g in gestionnaires-nord exploitation-prod \
         developpeurs equipe-web audit; do
  openstack group create $g --domain nord
done

# Étape 2 — délégation du domaine
openstack role add manager --domain nord \
  --group gestionnaires-nord --group-domain nord

# Étape 3 — audit : lecture du domaine et des projets
openstack role add reader --domain nord \
  --group audit --group-domain nord
openstack role add reader --domain nord --inherited \
  --group audit --group-domain nord
```

```
# Étape 4 — production : exploitation et développeurs
openstack role add member --inherited \
  --project prod --project-domain nord \
  --group exploitation-prod --group-domain nord
openstack role add reader --inherited \
  --project prod --project-domain nord \
  --group developpeurs --group-domain nord

# Étape 5 — hors production : affectations directes
for p in rec-web dev-web; do
  openstack role add member \
    --project $p --project-domain nord \
    --group equipe-web --group-domain nord
done

# Étape 6 — les personnes, dans les groupes
for g in developpeurs equipe-web; do
  openstack group add user --group-domain nord \
    --user-domain nord $g alice
done

# Contrôle
openstack role assignment list --names \
  --effective --user alice --user-domain nord
```

> Domaine raccordé à un annuaire (`sunbeam ldap add-domain`) : les groupes viennent de l'annuaire ; on saute les étapes 1 et 6.

Pas à pas (2) : automates et vérification

```
# Étape 7 — utilisateur technique, titulaire des automates
openstack user create ci-web --domain nord \
  --password-prompt \
  --description "Titulaire des automates web — equipe-web"
openstack role add member --user ci-web \
  --user-domain nord --project prod-web --project-domain nord

# Étape 8 — l'identifiant du pipeline, créé PAR ci-web
export OS_USERNAME=ci-web OS_USER_DOMAIN_NAME=nord
read -rs OS_PASSWORD; export OS_PASSWORD
export OS_PROJECT_NAME=prod-web OS_PROJECT_DOMAIN_NAME=nord
openstack application credential create deploy-web-2026q4 \
  --role member --expiration 2027-01-31T00:00:00
# → id et secret : dans le coffre du pipeline

# Étape 9 — le mot de passe de ci-web : au coffre aussi,
# utilisé seulement pour la rotation trimestrielle
```

Étape 10 — essai	Résultat attendu
Développeur : lister en production	accepté
Développeur : créer en production	403, politique
Développeur : créer en recette	accepté
Auditeur : lister les comptes du domaine	accepté
Auditeur : modifier un groupe de sécurité	403, politique
Gestionnaire : attribuer `admin`	403, politique
Gestionnaire : lire le domaine sud	403, politique
Pipeline : créer dans `prod-web`	accepté
Pipeline : agir dans `prod-bases`	refusé : autre projet

Si un refus attendu ne se produit pas, le cloud n'applique pas les nouvelles règles par défaut dans ce service : revenir à la slide « Secure RBAC sur votre cloud Sunbeam : vérifier ».

Auditer les droits

```
# Qui est admin, et où ?
openstack role assignment list --role admin --names

# Affectations faites à des personnes (pas à des groupes)
openstack role assignment list --names -f csv \
  -c User -c Role -c Project -c Domain | grep -v '^""'

# Tout ce qu'un compte obtient réellement
openstack role assignment list --effective --names \
  --user alice --user-domain nord

# Membres d'un groupe sensible
openstack user list --group exploitation-prod \
  --domain nord

# Identifiants d'application d'un compte technique
openstack application credential list --user ci-web

# Délégations en cours
openstack trust list
```

Contrôle	Seuil d'alerte
Nombre de comptes `admin`	plus de cinq personnes ; tout compte non nommé
`admin` hors du projet d'administration	toute occurrence
`service` détenu par une personne	toute occurrence
Affectations directes à des personnes	toute occurrence non justifiée
Identifiants sans expiration	toute occurrence
Comptes inactifs depuis 90 jours	à désactiver
Groupe sensible sans revue récente	plus d'un trimestre

> **Fréquence :** `admin` chaque mois ; le reste chaque trimestre ; après tout départ, le jour même.

La colonne utilisateur est vide pour les affectations de groupe ; le filtre proposé est indicatif et dépend du format de sortie du client.

5 · MISE EN ŒUVRE

Erreurs fréquentes

Erreur	Conséquence	Correction
`admin` sur un projet « pour qu'il puisse gérer son projet »	il administre tout le cloud	`member` ; `manager` si le service le prévoit
`admin` sur un domaine pour déléguer	même effet, par ses jetons de projet	`manager` sur le domaine
`admin-openrc` partagé	aucune imputabilité, fuite inévitable	comptes nominatifs, fichier gardé par la plateforme
Compte personnel dans un pipeline	tous les droits de la personne ; arrêt à son départ	utilisateur technique + identifiant d'application
Croire que `reader` protège sans vérifier	écriture possible si les nouvelles règles sont désactivées	tester avec un compte `reader`
Jeton système pour gérer des instances	403 dans Nova	jeton de projet
Rôle maison sans surcharge	le rôle ne donne rien	rôle standard, ou surcharge documentée
Modifier un compte de service OpenStack	services en panne, charms désynchronisés	ne jamais y toucher

Le piège principal tient en une phrase : dans les services de ressources, `admin` sur un projet n'est pas « administrateur de ce projet » ; c'est « administrateur du cloud ».

Exercices de mise en pratique

#	Énoncé
1	**Créer un compte `lecteur` avec `reader` sur un projet. Vérifier qu'il liste les instances et qu'il ne peut ni en créer ni ouvrir de console.**
2	**Déléguer le domaine `nord` à un gestionnaire. Avec son jeton de domaine : créer un compte et un projet, puis tenter d'attribuer `admin`. Relever le message.**
3	**Avec le même gestionnaire, tenter `openstack server list`. Expliquer le résultat, puis obtenir le droit de lister les instances d'un projet du domaine.**
4	**Créer un identifiant d'application `reader` qui expire dans une heure ; l'utiliser via `clouds.yaml` ; le révoquer ; constater l'effet.**
5	**Créer un identifiant restreint par règle d'accès à la seule liste des instances ; vérifier qu'il ne peut pas lister les réseaux.**
6	**Produire la liste des comptes `admin` et des détenteurs de `service` ; justifier chaque ligne.**

Corrigé — points clés

- **1 :** console refusée — règle de type `member` ; attention si la création réussit : nouvelles règles inactives.
- **2 :** 403 sur `admin` : il ne figure pas dans la liste des rôles attribuables par un `manager`.
- **3 :** un jeton de domaine n'ouvre aucune ressource. Il faut un rôle sur le projet — `reader` suffit — et un jeton de portée projet.
- **4 :** après suppression, une nouvelle authentification échoue (401) ; un jeton déjà obtenu fonctionne jusqu'à son expiration.
- **5 :** Neutron répond 401 : en validant le jeton, Keystone constate que cet appel ne figure pas dans les règles d'accès.
- **6 :** attendu — l'équipe plateforme pour `admin` ; les comptes des services, et eux seuls, pour `service`.

> Prérequis du TP : cloud Sunbeam opérationnel, `admin-openrc`, domaine `nord` et un projet contenant au moins une instance.

SYNTHÈSE

Ce qu'il faut retenir

Les rôles standards

- `reader` lit, `member` agit, `manager` gère, `admin` administre.
- `service` est réservé aux services, hors hiérarchie.
- Implications : une affectation suffit.
- Rôle maison = surcharge de politique ; en dernier recours.

Secure RBAC

- Trois portées : système, domaine, projet ; un jeton, une portée.
- Règle = portée acceptée + rôle + cible dans la portée.
- Deux options par service, à vrai par défaut depuis 2024.2.
- `admin` de projet reste global : ne jamais le déléguer.

Les comptes de service

- Comptes des services : gérés par les charms, sensibles, intouchables.
- Automates : identifiant d'application, rôle minimal, un projet, une date de fin.
- Règles d'accès pour restreindre aux appels utiles.
- Rotation sans coupure, révocation immédiate.

Question	Réponse courte
Déléguer un domaine ?	`manager` **sur le domaine, à un groupe**
Donner la lecture seule ?	`reader` — après avoir vérifié par un essai que le service l'applique
Un automate ?	utilisateur technique + identifiant d'application `member` ou `reader`, expirant, dans un coffre
Sur Sunbeam ?	rôles standards présents ; configuration des politiques écrite par les charms : la lire, la tester, ne pas l'éditer dans les pods

Sources : documentation de Keystone (protection des API et rôles par défaut, identifiants d'application), objectif communautaire « Consistent and Secure Default RBAC », documentation de Nova (concepts de politique), notes de version d'oslo.policy 2024.2, dépôt des charms Sunbeam — consultés en octobre 2026.